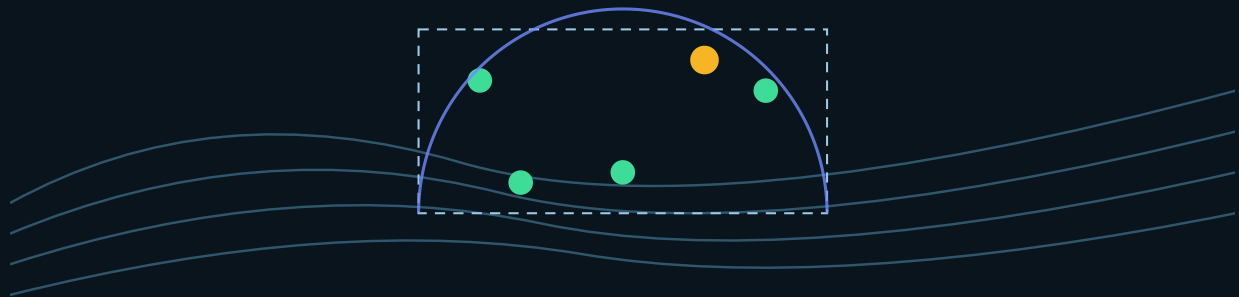


ARGUS

Rapport d'ingénierie technique
Variante C « Jumeau virtuel » sécurisé



Groupe

Consortium

Filières

Version

Gn

à compléter

EISI DEV · EISI IA · EISI INFRA

v1.0 — 06/10/2026

Sommaire

1. Synthèse
2. Variante d'architecture retenue
3. Schéma réseau et plan d'adressage
4. Flux de données et protocole
5. Schéma de câblage électronique (nœud ESP32)
6. Matrice de sécurité
7. Documentation de l'intelligence artificielle
8. Rapport d'audit post-pentest
9. Maintien en condition opérationnelle
10. Annexe : poster A3

1. Synthèse

ARGUS supervise une flotte de nœuds Sentinel-X virtuels déployés sur l'avant-poste AetherCorp K-7. Il réunit sur une seule chronologie les trois menaces du cahier des charges : **environnementale** (fuite de gaz, surchauffe), **intrusion physique** et **cyberattaque**.

32 s

détection d'une surchauffe lente par l'IA, contre 278 s pour un seuil fixe

6 / 6

attaques de l'audit bloquées et détectées sur la pile réelle

7

couches de défense en profondeur, de l'hôte à la détection

22

tests automatisés (signatures, rejeu, usurpation, rôles, corrélation)

Ce que le jury voit en démonstration

- Un jumeau numérique 3D de l'avant-poste : chaque message d'un nœud y vole jusqu'au mât de communication ; une fuite de gaz produit une fumée verte, une surchauffe une zone rouge, un intrus apparaît et est suivi par la caméra du mât.
- Un nœud « héros » exécutant un vrai micrologiciel C++ sur un ESP32 émulé (Wokwi), relié en TLS mutuel au serveur.
- Chaque attaque de l'audit est rejetée par la couche prévue et apparaît en direct sur la chronologie cyber.

Écarts assumés par rapport au cahier des charges

Cahier des charges	ARGUS	Justification
Boîtier physique ESP8266	ESP32 émulé (Wokwi) + flotte simulée	Variante validée par notre référente ; ESP8266 non émulé par Wokwi ; l'ESP32 accélère AES/SHA/ECC pour le TLS. Le même binaire se flashe sur un ESP32 réel.
Boîtier imprimé 3D, gravure laser	Non réalisé	Pas de matériel électronique ; validé avec notre référente. Le produit présenté est la plateforme.
Webcam USB + YOLO	Script YOLOv8n fourni ; caméra virtuelle du jumeau en démonstration	Le script réel (≈35 ms/trame) reste livré et peut alimenter l'API.

2. Variante d'architecture retenue

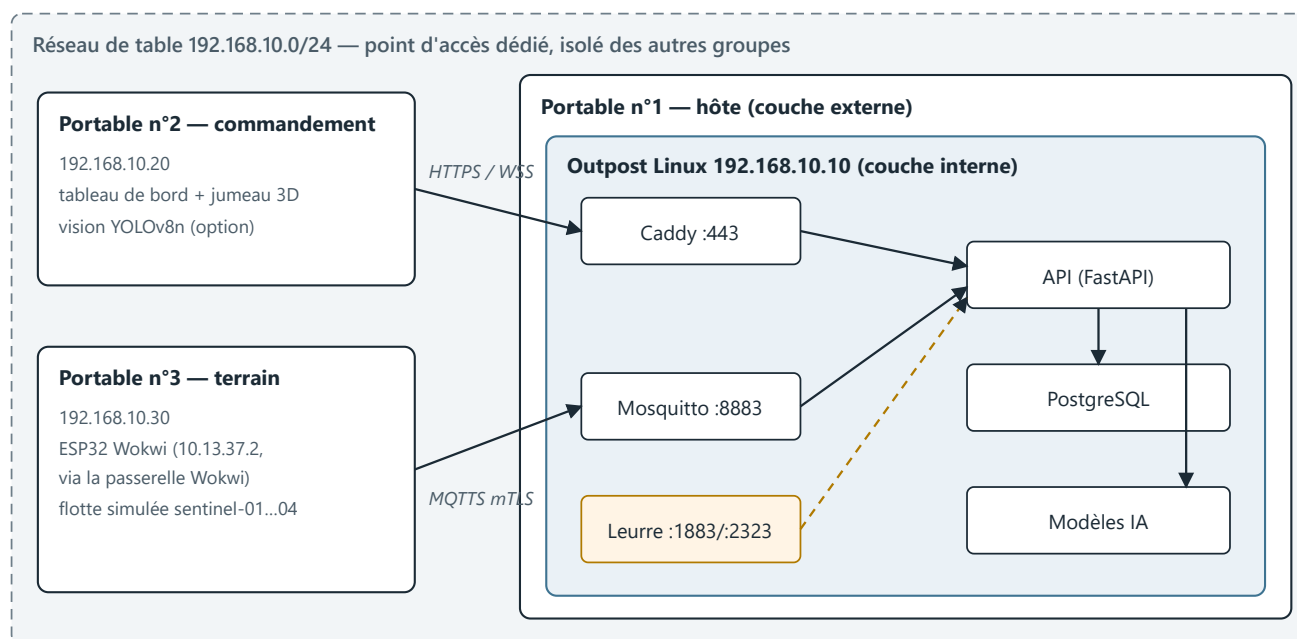
Le cahier des charges propose deux options (A : Raspberry Pi 5 ; B : portable d'un apprenant). ARGUS retient une **option C « jumeau virtuel »** : un portable héberge une **machine Linux isolée** (« Outpost ») qui exécute toute la pile Docker-Compose. C'est la « double couche » demandée par notre référente : un système sécurisé à l'intérieur d'un système sécurisé.

Couche	Rôle	Réalisation
Externe : poste hôte	N'expose rien directement	Windows 11, pare-feu, WSL2 / VM
Interne : Outpost Linux	Exécute la pile, seule surface réseau	Ubuntu 24.04, Docker Engine, UFW
Conteneurs	Isolés par réseau et par privilèges	5 services + profils optionnels

Composants

Composant	Technologie	Filière
Nœud héros (micrologiciel)	C++ / PlatformIO, ESP32 émulé (Wokwi)	DEV
Flotte simulée + scénarios	Python, modèle physique des capteurs	DEV
Broker	Eclipse Mosquitto 2.0, MQTT sur TLS mutuel	INFRA
API REST + WebSocket	FastAPI, PostgreSQL 16	DEV
Tableau de bord	React 19, Three.js (jumeau 3D), Recharts	DEV
Proxy HTTPS	Caddy 2	INFRA
Détection d'anomalies	scikit-learn (Isolation Forest, EllipticEnvelope)	IA
Vision	YOLOv8n / OpenCV HOG, floutage des visages	IA
Leurre (honeypot)	Python asyncio	INFRA

3. Schéma réseau et plan d'adressage



Topologie « Edge-to-Server » à trois postes. Repli possible sur un seul portable (même architecture, IP différentes).

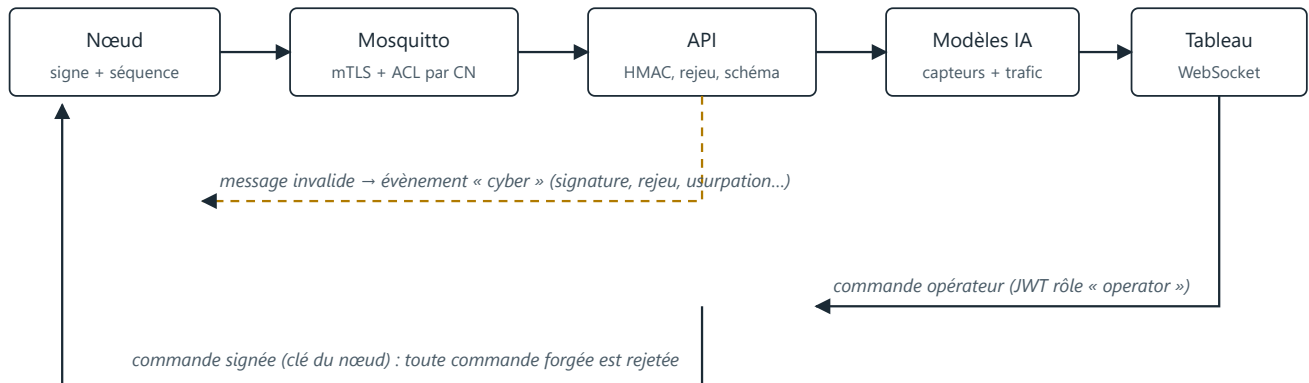
Plan d'adressage

Segment	Plage	Hôtes
LAN de table (Wi-Fi)	192.168.10.0/24	.1 AP · .10 Outpost · .20 commandement · .30 terrain · .100–.150 DHCP invités
Docker <code>edge_net</code>	172.20.10.0/24	mosquitto, leurre (seuls services publiés)
Docker <code>core_net</code> (interne)	172.20.20.0/24	api, postgres, mosquitto — aucune route vers l'extérieur
Docker <code>ops_net</code>	172.20.30.0/24	caddy, api
Docker <code>decoy_net</code> (interne)	172.20.40.0/24	leurre → api (alertes uniquement)
Wi-Fi virtuel Wokwi	10.13.37.0/24	nœud héros, NAT par la passerelle Wokwi

Surface exposée de l'Outpost

Port	Service	Autorisé depuis
443/tcp	Caddy (tableau de bord + API)	LAN de table
8883/tcp	Mosquitto, TLS mutuel	LAN de table
1883/tcp, 2323/tcp	Leurre (faux broker, fausse console)	tous — chaque accès est journalisé
22/tcp	SSH par clé uniquement	poste d'administration 192.168.10.20

4. Flux de données et protocole



Topics MQTT

Topic	Sens	QoS	Contenu
argus/v1/nodes/{id}/telemetry	nœud → serveur	1	mesures toutes les 2 s
argus/v1/nodes/{id}/events	nœud → serveur	1	pir, boot, tamper, ack
argus/v1/nodes/{id}/status	nœud → serveur	1 (retenu)	online / offline (Last Will)
argus/v1/nodes/{id}/cmd	serveur → nœud	1	buzzer, led, écran

{id} est le Common Name du certificat client du nœud : Mosquitto l'impose (`use_identity_as_username` + ACL `pattern`), un nœud ne peut donc jamais publier au nom d'un autre.

Enveloppe signée

```
{"v":1,"node":"sentinel-01","seq":1759700000123,"ts":1759700000,"type":"telemetry",
  "data":{"temp_c":24.1,"hum_pct":41.0,"gas_ppm":180,"pir":false}}
<HMAC-SHA256(clé_du_nœud, octets_exacts_du_corps) en hexadécimal>
```

- **Anti-rejeu** : `seq` strictement croissant par nœud ; tout `seq` déjà vu est rejeté.
- **Fraîcheur** : écart d'horloge > 30 s rejeté (le nœud se synchronise en NTP).
- **Pourquoi mTLS et HMAC ?** mTLS protège chaque saut ; l'HMAC protège de bout en bout à *travers* le broker : un broker compromis ne peut ni forger ni altérer une mesure.

API REST / WebSocket (<https://outpost/api/v1>)

Méthode	Chemin	Accès	Rôle
POST	/auth/login	public, limité en débit	obtenir un JWT
POST	/alerts	jeton de service	alertes externes (vision, leurre) — exigé par le sujet
GET · WS	/nodes, /telemetry, /events · /ws	viewer	flotte, historiques, chronologie, flux temps réel
POST	/nodes/{id}/commands	operator	actionneurs

5. Schéma de câblage électronique (nœud ESP32)

Montage défini dans `firmware/sentinel-node/diagram.json` et simulé par Wokwi. Les composants sont ceux de la liste de matériel du sujet (DHT22, MQ-2, PIR HC-SR501, OLED I2C 0,96", buzzer, LED) plus un contact d'ouverture du boîtier.



Le MQ-2 est alimenté en 3,3 V pour rester dans la plage de l'ADC ; l'entrée TAMPER utilise le pull-up interne (actif à l'état bas).

Composant	Broches ESP32	Alimentation	Usage dans le micrologiciel
DHT22	GPIO15	3V3	température / humidité toutes les 2 s
MQ-2	GPIO34 (ADC1)	3V3	courbe log-log Rs/R0 ; R0 calibré après stabilisation (≤30 s), aucune valeur envoyée avant
PIR HC-SR501	GPIO27	5V	front montant → évènement « pir »
Bouton TAMPER	GPIO4 (pull-up)	—	ouverture du boîtier → évènement « tamper »
OLED SSD1306	GPIO21/22 (I2C)	3V3	IP, état TLS, mesures, dernier évènement, message opérateur
Buzzer	GPIO25	—	sirène pilotée à distance (commande signée)
LED ×3	GPIO18/19/23 + 220 Ω	—	état : vert normal, orange connexion, rouge alarme

Ressources : 74,6 % de la flash et 14,4 % de la RAM d'un ESP32 DevKit. Clés et certificats injectés à la compilation depuis un en-tête généré (`secrets.h` , jamais versionné).

6. Matrice de sécurité

Couche	Mesure	Menace couverte	Preuve
Hôte	Pare-feu Windows, seule la machine Outpost est exposée	Accès direct à l'hôte	capture des règles
Système Outpost	UFW : refus par défaut, 4 ports autorisés	Découverte de services	<code>ufw status verbose</code> , nmap depuis un autre poste
	SSH par clé uniquement, pas de root, utilisateur unique, fail2ban	Force brute SSH	<code>sshd -T</code>
	sysctl réseau durci, auditd, mises à jour automatiques	Usurpation IP, persistance	<code>/etc/sysctl.d/90-argus.conf</code>
Conteneurs	Système de fichiers en lecture seule, <code>cap_drop: ALL</code> , <code>no-new-privileges</code> , utilisateurs non-root, limites mémoire/CPU/PID	Évasion, déni de service local	<code>docker inspect</code> , <code>compose</code>
	Réseaux internes sans route sortante (<code>core_net</code> , <code>decoy_net</code>)	Rebond latéral	<code>docker network inspect</code>
Transport	MQTTS 8883 avec certificat client obligatoire (mTLS), TLS ≥ 1.2	Écoute, homme du milieu, client non autorisé	Wireshark : trafic chiffré uniquement
	HTTPS 443 (Caddy), en-têtes CSP / X-Frame-Options / nosniff	Interception, XSS, clickjacking	<code>curl -I</code>
	AC privée avec contraintes de nom : ne peut signer que pour les noms ARGUS	Abus de l'AC si sa clé fuit	certificat « <code>www.mybank.com</code> » rejeté : <i>permitted subtree violation</i>
Identité / message	Un certificat par nœud, ACL Mosquitto liées au CN	Usurpation d'un nœud	test d'audit « usurpation »
	HMAC-SHA256 par nœud, séquence anti-rejeu, fraîcheur 30 s, validation de schéma, taille max	Falsification, rejeu, injection	tests d'audit + 22 tests unitaires
Application	JWT, rôles viewer / operator, mots de passe crypt	Élévation de privilèges	test : viewer → 403 sur les commandes
	Limitation de débit (login, alertes, commandes), jetons de service	Force brute, inondation	test : 429 après 5 essais
Détection	Chaque message rejeté devient un évènement « cyber »	Attaque silencieuse	chronologie du tableau de bord
	Modèle de trafic (EllipticEnvelope), surveillance du journal Mosquitto	Inondation, prise de session	audit : détection confirmée
	Leurre : faux broker et fausse console	Reconnaissance, vol d'identifiants	identifiants capturés en direct
Secrets	Docker secrets, aucun secret dans Git (<code>.gitignore</code> vérifié)	Fuite par le dépôt	contrôle avant chaque commit

7. Documentation de l'intelligence artificielle

Modèle capteurs : Isolation Forest

Conformément au sujet, aucune règle statique du type `if temp > 40` n'est utilisée pour détecter. Chaque nœud alimente une fenêtre glissante de 15 mesures (30 s) transformée en 10 caractéristiques : niveaux, **pentés** (cinétique), volatilité, **corrélacion température/gaz** et taux de présence. Le modèle est entraîné uniquement sur du fonctionnement normal (44 580 fenêtres issues de 30 nœuds simulés avec des lignes de base différentes) ; il signale ce qui s'en écarte.

Scénario	Détection par le modèle	Seuil fixe (référence)	Gain
Fuite de gaz	4 s	58 s	×14
Surchauffe lente + micro-déviacion de gaz (exemple du sujet)	32 s	278 s	×8,7
Capteur bloqué / erratique	2 s	212 s	×106

Taux de fausses alertes sur données normales : **0,34 %**. Le seuil n'existe que comme vérité terrain pour l'évaluation.

Modèle trafic : EllipticEnvelope

Fenêtres de 10 s par nœud : nombre de messages, taille moyenne, messages rejetés, types distincts (échelle logarithmique). **Retour d'expérience** : un Isolation Forest a d'abord été essayé et a échoué. Il ne sait pas extrapoler : une inondation de 400 messages tombe dans la même feuille que le seau normal le plus chargé. Une distance de Mahalanobis robuste (EllipticEnvelope) croît sans borne avec l'écart et détecte tout : inondations de 30 et 400 messages, charges anormales, messages rejetés, nœud devenu silencieux — 0 % de fausses alertes. Un délai de grâce au redémarrage évite de confondre la rafale de reconnexion avec une attaque ; il est plafonné, car l'audit a montré qu'un nœud compromis pouvait sinon masquer une inondation derrière un faux « boot ».

Corrélacion

Les sorties des modèles et des capteurs sont pondérées sur 30 s (présence 1, personne vue par la caméra 2, anomalie capteur 2, évènement cyber 1, leurre 1). Score ≥ 3 : incident « warning » ; ≥ 4 : « critical ». Exemple : mouvement + personne + attaque réseau sur le même nœud → incident critique.

Vision

- Script autonome `ai/vision/vision.py` : webcam USB redimensionnée en 640×480, YOLOv8n (classe « personne »), repli OpenCV HOG sans GPU (≈ 35 ms par trame, sous la contrainte de 100 ms du sujet).
- Zone interdite polygonale : une personne dont les pieds entrent dans la zone déclenche une alerte (anti-rebond 10 s) via `POST /api/v1/alerts`, vérifiée en TLS contre l'AC ARGUS.
- Protection des données (RGPD)** : la tête de chaque personne est pixelisée *avant* tout affichage ou diffusion ; aucune image n'est enregistrée.

8. Rapport d'audit post-pentest

Périmètre et méthode

Audit de **notre propre pile**, sur l'environnement réel (Mosquitto, Caddy, PostgreSQL dans l'Outpost), avec l'outil d'auto-contrôle du dépôt (`tools/redteam/attacks.py`). Pour chaque scénario, deux questions : l'attaque est-elle **bloquée** ? et le tableau de bord l'a-t-il **détectée** (évènement confirmé via l'API) ?

Résultats (pile réelle, 6 octobre 2026)

Scénario	Bloqué par	Détecté	Évènement
Connexion sans certificat client	Mosquitto (mTLS)	oui	<code>tls_rejected</code>
MQTT en clair sur 1883	c'est le leurre	oui	<code>decoy_mqtt_login</code> + identifiants
Usurpation d'un autre nœud	ACL Mosquitto	oui	<code>session_takeover</code>
Message signé altéré	API (HMAC)	oui	<code>bad_signature</code>
Rejeu d'un message capturé	API (séquence)	oui	<code>replay</code>
Inondation (400 messages)	limites Mosquitto + IA	oui	<code>traffic_anomaly</code>

Constats corrigés pendant l'audit

#	Constat	Gravité	Correction
1	Les alertes du leurre étaient refusées (401) : jetons de service mal concaténés à l'installation	Élevée (détection aveugle)	script corrigé ; l'API accepte virgule ou retour à la ligne
2	Un faux évènement « boot » ouvrait un délai de grâce masquant une inondation	Élevée (évasion)	grâce plafonnée aux petites rafales ; test de non-régression
3	L'API ne pouvait pas lire le journal Mosquitto : détections côté broker inopérantes	Moyenne	journal lisible par le seul groupe de l'API ; lecture résiliente
4	Configuration et ACL Mosquitto lisibles par tous	Moyenne	fichiers privés à l'utilisateur mosquitto (0600)
5	La prise de session d'un nœud n'était pas signalée	Moyenne	nouvelle détection <code>session_takeover</code>
6	AC racine sans contraintes, à importer dans le magasin Windows	Moyenne	AC réémise avec contraintes de nom
7	Les anciennes fausses alertes de rejeu (deux messages dans la même milliseconde)	Faible	séquence strictement croissante par émetteur

Risques résiduels

- Une publication refusée par les ACL est bloquée mais **silencieuse** si l'attaquant utilise un identifiant client différent : Mosquitto ne journalise ces refus qu'en mode verbeux. Choix assumé (volume de journaux) ; l'attaque reste sans effet.
- La clé privée de l'AC réside sur le poste d'administration ; les contraintes de nom en limitent l'impact.
- Le tableau de bord de développement n'est pas exposé ; seul Caddy l'est.

Pentest croisé du jeudi

Heure	Source	Technique observée	Bloqué par	Détecté

À compléter le jeudi à partir de la chronologie cyber et des identifiants capturés par le leurre.

9. Maintien en condition opérationnelle

Sujet	Mesure
Volume de journaux	rotation Docker 3 × 10 Mo par conteneur ; journaux Caddy roulés à 10 Mo ; journaux Mosquitto filtrés (pas de mode verbeux)
Ressources	limites mémoire, CPU et PID par conteneur ; Outpost plafonné à 4 Go de RAM ; profil <code>monitoring</code> (Netdata) pour CPU/RAM hôte et conteneurs
Disponibilité	<code>restart: unless-stopped</code> , contrôles de santé (API, PostgreSQL), démarrage automatique de la pile avec l'Outpost
Supervision des nœuds	états online / silencieux / offline (Last Will), alerte si un nœud se tait plus de 15 s
Redéploiement	une commande (<code>infra/wsl/setup-outpost-wsl.sh</code>), idempotente ; secrets conservés
PKI	<code>pki.sh bootstrap</code> réémet l'ensemble ; liste de révocation chargée par Mosquitto

ARGUS

Un avant-poste surveillé par cent yeux : capteurs, caméra et réseau sur une seule chronologie

Trois menaces, une réponse

Fuite de gaz et surchauffe, intrusion physique, cyberattaque : ARGUS les détecte et les corrèle. Mouvement + personne vue + attaque réseau sur le même point = incident critique.

×8,7

plus rapide qu'un seuil fixe pour détecter une surchauffe lente (32 s au lieu de 278 s)

Double couche

La pile tourne dans une machine Linux isolée, à l'intérieur du poste hôte. Seuls 443, 8883 et le leurre sont visibles.

7 couches de défense

1 Hôte

2 Système Outpost

3 Conteneurs

4 Transport mTLS

5 Message signé

6 Application

7 Détection IA + leurre

6 / 6

attaques de l'audit bloquées et détectées en direct

Nœud réel, sans matériel

Micrologiciel C++ sur ESP32 émulé : DHT22, MQ-2, PIR, OLED, sirène. Le même binaire se flashe sur une carte réelle.

IA explicable

Isolation Forest sur la cinétique des capteurs, Mahalanobis sur le trafic réseau, YOLOv8n avec floutage des visages.

Jumeau numérique 3D

Chaque message vole jusqu'au mât ; gaz en fumée verte, surchauffe en zone rouge, intrus suivi par la caméra.

Sentinel-X : la sécurité à la bordure.